

I build compiler infrastructure where composition, IR/runtime decisions, and correctness are explicit: LLVM analyses and passes, deterministic language-platform planning, Bytecode/AIR/SSA, backends, and independent verifier/oracle checks

LLVM analysis

LICM plus interprocedural linear summaries for global values; unsupported CFG/call cases conservatively fall back to unknown

Language infrastructure

UniversalToolchain: one immutable LanguagePlan owns dependencies, capabilities, pass ordering, and backend routes

Independent verification

Interpreter/CIL parity, exact package binding, allocator verification, and differential execution against a reference interpreter

EXPERIENCE

July 1 — August 31, 2026

MCST — Compiler Engineering Intern

- Implemented an LLVM LICM pass and am developing interprocedural global-value evolution analysis: linear summaries compose where justified, while unsupported loops/calls/CFG cases fall back to unknown
- Validate transformations with opt/lli, verifier checks, and negative cases where the pass must preserve the original IR or decline an inference

2024 — present

UniversalToolchain — creator and primary developer

- Made LanguageCompiler the single owner of composition: dependencies/ conflicts, capability selection, pass ordering, and backend routes are fixed in an immutable LanguagePlan before runtime
- Separated syntax → semantic binding → Bytecode → AIR → optimization → backend through explicit representation boundaries; exact binding and regression tests enforce the contracts, with a current exact manifest of 1,306/1,306 passing

2026

x86-64 Codegen & Register Allocation Playground

- Separated allocation strategy from correctness policy: allocators return a common Assignment that an independent verifier checks against liveness/ interference invariants before code generation
- Differentially compare native x86-64 execution with a reference interpreter in a separate resource-bounded process

SKILLS

LLVM / Program Analysis

C++23, LLVM passes, LoopInfo, CFG, dominance, SSA, interprocedural summaries, conservative analysis

Compiler Infrastructure

typed language packages, deterministic planning, capabilities/ conflicts, pass ordering, exact runtime binding

IR / Runtime / Backend

NET, Bytecode/AIR, SSA routes, interpreter, CIL backend, register allocation, x86-64

Verification / Tooling

reference oracles, differential/ metamorphic tests, verifier-gated transforms, CMake, Linux, ASan/UBSan

EDUCATION

Software Engineering student at HSE University

RECOGNITION

First-degree diploma and the Grand Prize “Perfection as Hope” for UniversalToolchain

Moscow / remote

PROJECTS

UniversalToolchain / Wist2

The current exact test manifest passes with zero failures; interpreter/CIL parity, clean-consumer scenarios, and reproducible compiler/runtime contracts are verified.

x86-64 Codegen & Register Allocation Lab

A measurable pipeline with spill/reload metrics, code size, and reproducible result comparison.

PS-form Memory Dependence Analyzer

Conservative yes/no/maybe results checked by an independent exact oracle on small domains, randomized/ metamorphic tests, and reproducible counterexamples.